

10 Logistic Regression

10.1 Introduction

Logistic regression is a widely used discriminative classification model $p(y|\mathbf{x};\boldsymbol{\theta})$, where $\mathbf{x} \in \mathbb{R}^D$ is a fixed-dimensional input vector, $y \in \{1, \dots, C\}$ is the class label, and $\boldsymbol{\theta}$ are the parameters. If $C = 2$, this is known as **binary logistic regression**, and if $C > 2$, it is known as **multinomial logistic regression**, or alternatively, **multiclass logistic regression**. We give the details below.

10.2 Binary logistic regression

Binary logistic regression corresponds to the following model

$$p(y|\mathbf{x}, \boldsymbol{\theta}) = \text{Ber}(y|\sigma(\mathbf{w}^\top \mathbf{x} + b)) \quad (10.1)$$

where σ is the sigmoid function defined in Section 2.4.2, $\mathbf{w}$ are the weights, b is the bias, and $\boldsymbol{\theta} = (\mathbf{w}, b)$ are all the parameters. In other words,

$$p(y = 1|\mathbf{x}, \boldsymbol{\theta}) = \sigma(a) = \frac{1}{1 + e^{-a}} \quad (10.2)$$

where $a = \mathbf{w}^\top \mathbf{x} + b = \log(\frac{p}{1-p})$, is the log-odds, and $p = p(y = 1|\mathbf{x}, \boldsymbol{\theta})$. (In ML, the quantity a is usually called the **logit** or the **pre-activation**.)

Sometimes we choose to use the labels $\tilde{y} \in \{-1, +1\}$ instead of $y \in \{0, 1\}$. We can compute the probability of these alternative labels using

$$p(\tilde{y}|\mathbf{x}, \boldsymbol{\theta}) = \sigma(\tilde{y}a) \quad (10.3)$$

since $\sigma(-a) = 1 - \sigma(a)$. This slightly more compact notation is widely used in the ML literature.

10.2.1 Linear classifiers

The sigmoid gives the probability that the class label is $y = 1$. If the loss for misclassifying each class is the same, then the optimal decision rule is to predict $y = 1$ iff class 1 is more likely than class 0, as we explained in Section 5.1.2.2. Thus

$$f(\mathbf{x}) = \mathbb{I}(p(y = 1|\mathbf{x}) > p(y = 0|\mathbf{x})) = \mathbb{I}\left(\log \frac{p(y = 1|\mathbf{x})}{p(y = 0|\mathbf{x})} > 0\right) = \mathbb{I}(a > 0) \quad (10.4)$$

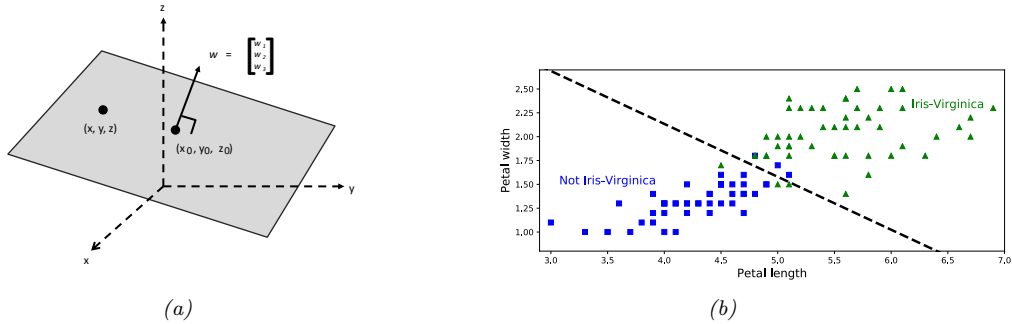


Figure 10.1: (a) Visualization of a 2d plane in a 3d space with surface normal $\mathbf{w}$ going through point $\mathbf{x}_0 = (x_0, y_0, z_0)$. See text for details. (b) Visualization of optimal linear decision boundary induced by logistic regression on a 2-class, 2-feature version of the iris dataset. Generated by `iris_logreg.ipynb`. Adapted from Figure 4.24 of [Gér19].

where $a = \mathbf{w}^\top \mathbf{x} + b$.

Thus we can write the prediction function as follows:

$$f(\mathbf{x}; \boldsymbol{\theta}) = b + \mathbf{w}^\top \mathbf{x} = b + \sum_{d=1}^D w_d x_d \quad (10.5)$$

where $\mathbf{w}^\top \mathbf{x} = \langle \mathbf{w}, \mathbf{x} \rangle$ is the inner product between the weight vector $\mathbf{w}$ and the feature vector $\mathbf{x}$. This function defines a linear **hyperplane**, with normal vector $\mathbf{w} \in \mathbb{R}^D$ and an offset $b \in \mathbb{R}$ from the origin.

Equation (10.5) can be understood by looking at Figure 10.1a. Here we show a plane in a 3d feature space going through the point $\mathbf{x}_0$ with surface normal $\mathbf{w}$. Points on the surface satisfy $\mathbf{w}^\top (\mathbf{x} - \mathbf{x}_0) = 0$. If we define $b = -\mathbf{w}^\top \mathbf{x}_0$, we can rewrite this as $\mathbf{w}^\top \mathbf{x} + b = 0$. This plane separates 3d space into two **half spaces**. This linear plane is known as a **decision boundary**. If we can perfectly separate the training examples by such a linear boundary (without making any classification errors on the training set), we say the data is **linearly separable**. From Figure 10.1b, we see that the two-class, two-feature version of the iris dataset is not linearly separable.

In general, there will be uncertainty about the correct class label, so we need to predict a probability distribution over labels, and not just decide which side of the decision boundary we are on. In Figure 10.2, we plot $p(y = 1 | (x_1, x_2), \mathbf{w}) = \sigma(w_1 x_1 + w_2 x_2)$ for different weight vectors $\mathbf{w}$. The vector $\mathbf{w}$ defines the orientation of the decision boundary, and its magnitude, $\|\mathbf{w}\| = \sqrt{\sum_{d=1}^D w_d^2}$, controls the steepness of the sigmoid, and hence the confidence of the predictions.

10.2.2 Nonlinear classifiers

We can often make a problem linearly separable by preprocessing the inputs in a suitable way. In particular, let $\phi(\mathbf{x})$ be a transformed version of the input feature vector. For example, suppose we use $\phi(x_1, x_2) = [1, x_1^2, x_2^2]$, and we let $\mathbf{w} = [-R^2, 1, 1]$. Then $\mathbf{w}^\top \phi(\mathbf{x}) = x_1^2 + x_2^2 - R^2$, so the decision boundary (where $f(\mathbf{x}) = 0$) defines a circle with radius R , as shown in Figure 10.3. The resulting

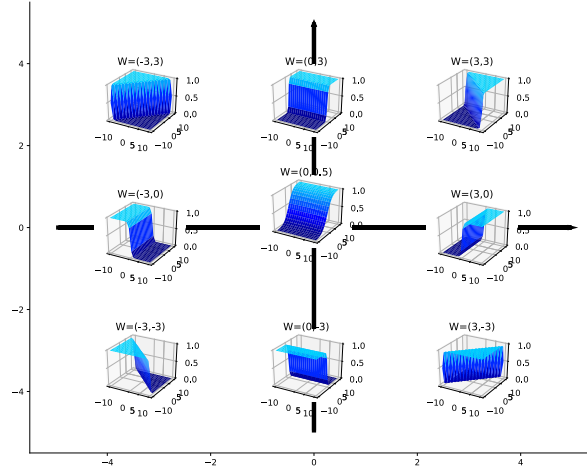


Figure 10.2: Plots of $\sigma(w_1x_1 + w_2x_2)$. Here $\mathbf{w} = (w_1, w_2)$ defines the normal to the decision boundary. Points to the right of this have $\sigma(\mathbf{w}^\top \mathbf{x}) > 0.5$, and points to the left have $\sigma(\mathbf{w}^\top \mathbf{x}) < 0.5$. Adapted from Figure 39.3 of [Mac03]. Generated by `sigmoid_2d_plot.ipynb`.

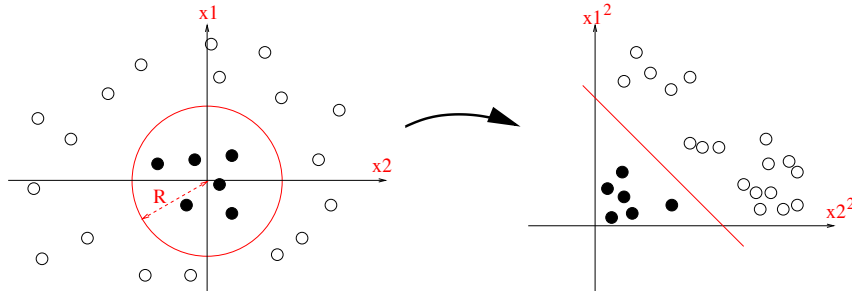


Figure 10.3: Illustration of how we can transform a quadratic decision boundary into a linear one by transforming the features from $\mathbf{x} = (x_1, x_2)$ to $\phi(\mathbf{x}) = (x_1^2, x_2^2)$. Used with kind permission of Jean-Philippe Vert.

function f is still linear in the parameters $\mathbf{w}$, which is important for simplifying the learning problem, as we will see in Section 10.2.3. However, we can gain even more power by learning the parameters of the feature extractor $\phi(\mathbf{x})$ in addition to linear weights $\mathbf{w}$; we discuss how to do this in Part III.

In Figure 10.3, we used a quadratic expansion of the features. We can also use a higher order polynomial, as in Section 1.2.2.2. In Figure 10.4, we show the effects of using polynomial expansion up to degree K on a 2d logistic regression problem. As in Figure 1.7, we see that the model becomes more complex as the number of parameters increases, and eventually results in overfitting. We discuss ways to reduce overfitting in Section 10.2.7.

10.2.3 Maximum likelihood estimation

In this section, we discuss how to estimate the parameters of a logistic regression model using maximum likelihood estimation.

10.2.3.1 Objective function

The negative log likelihood (scaled by the dataset size N) is given by the following (we assume the bias term b is absorbed into the weight vector $\mathbf{w}$):

$$\text{NLL}(\mathbf{w}) = -\frac{1}{N} \log p(\mathcal{D}|\mathbf{w}) = -\frac{1}{N} \log \prod_{n=1}^N \text{Ber}(y_n|\mu_n) \quad (10.6)$$

$$= -\frac{1}{N} \sum_{n=1}^N \log[\mu_n^{y_n} \times (1 - \mu_n)^{1-y_n}] \quad (10.7)$$

$$= -\frac{1}{N} \sum_{n=1}^N [y_n \log \mu_n + (1 - y_n) \log(1 - \mu_n)] \quad (10.8)$$

$$= \frac{1}{N} \sum_{n=1}^N \mathbb{H}_{ce}(y_n, \mu_n) \quad (10.9)$$

where $\mu_n = \sigma(a_n)$ is the probability of class 1, $a_n = \mathbf{w}^\top \mathbf{x}_n$ is the **logit**, and $\mathbb{H}_{ce}(y_n, \mu_n)$ is the **binary cross entropy** defined by

$$\mathbb{H}_{ce}(p, q) = -[p \log q + (1 - p) \log(1 - q)] \quad (10.10)$$

If we use $\tilde{y}_n \in \{-1, +1\}$ instead of $y_n \in \{0, 1\}$, then we can rewrite this as follows:

$$\text{NLL}(\mathbf{w}) = -\frac{1}{N} \sum_{n=1}^N [\mathbb{I}(\tilde{y}_n = 1) \log(\sigma(a_n)) + \mathbb{I}(\tilde{y}_n = -1) \log(\sigma(-a_n))] \quad (10.11)$$

$$= -\frac{1}{N} \sum_{n=1}^N \log(\sigma(\tilde{y}_n a_n)) \quad (10.12)$$

$$= \frac{1}{N} \sum_{n=1}^N \log(1 + \exp(-\tilde{y}_n a_n)) \quad (10.13)$$

However, in this book, we will mostly use the $y_n \in \{0, 1\}$ notation, since it is easier to generalize to the multiclass case (Section 10.3), and makes the connection with cross-entropy easier to see.

10.2.3.2 Optimizing the objective

To find the MLE, we must solve

$$\nabla_{\mathbf{w}} \text{NLL}(\mathbf{w}) = \mathbf{g}(\mathbf{w}) = \mathbf{0} \quad (10.14)$$

We can use any gradient-based optimization algorithm to solve this, such as those we discuss in Chapter 8. We give a specific example in Section 10.2.4. But first we must derive the gradient, as we explain below.

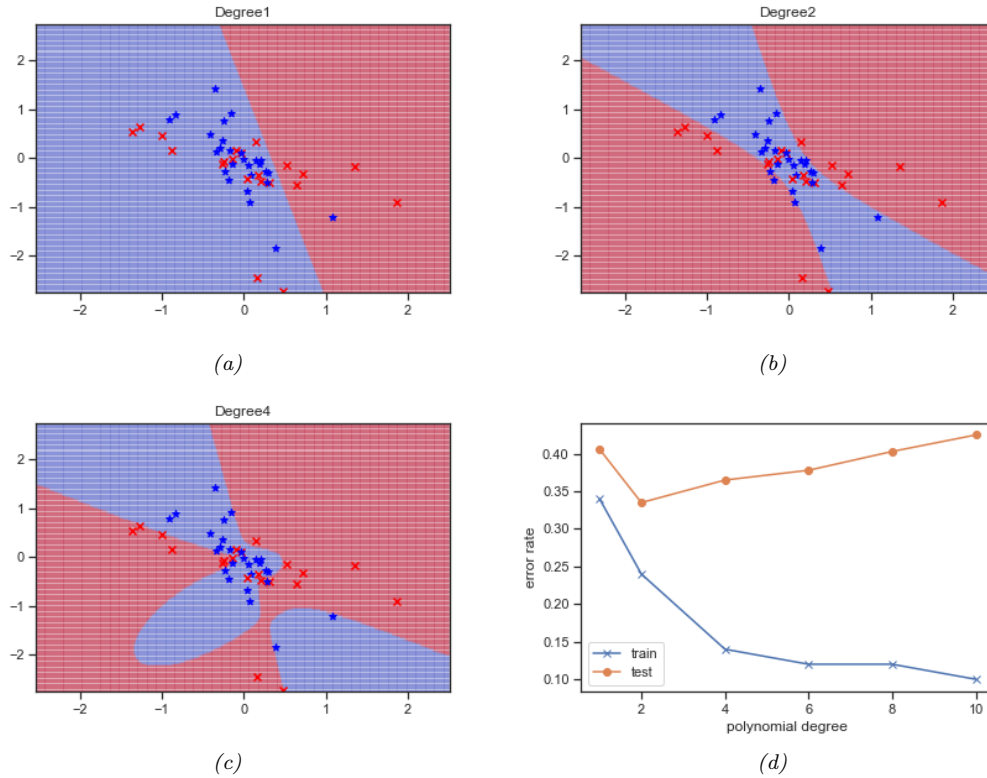


Figure 10.4: Polynomial feature expansion applied to a two-class, two-dimensional logistic regression problem. (a) Degree $K = 1$. (b) Degree $K = 2$. (c) Degree $K = 4$. (d) Train and test error vs degree. Generated by [logreg_poly_demo.ipynb](#).

10.2.3.3 Deriving the gradient

Although we can use automatic differentiation methods (Section 13.3) to compute the gradient of the NLL, it is also easy to do explicitly, as we show below. Fortunately the resulting equations will turn out to have a simple and intuitive interpretation, which can be used to derive other methods, as we will see.

To start, note that

$$\frac{d\mu_n}{da_n} = \sigma(a_n)(1 - \sigma(a_n)) \quad (10.15)$$

where $a_n = \mathbf{w}^\top \mathbf{x}_n$ and $\mu_n = \sigma(a_n)$. Hence by the chain rule (and the rules of vector calculus, discussed in Section 7.8) we have

$$\frac{\partial}{\partial w_d} \mu_n = \frac{\partial}{\partial w_d} \sigma(\mathbf{w}^\top \mathbf{x}_n) = \frac{\partial}{\partial a_n} \sigma(a_n) \frac{\partial a_n}{\partial w_d} = \mu_n(1 - \mu_n)x_{nd} \quad (10.16)$$

The gradient for the bias term can be derived in the same way, by using the input $x_{n0} = 1$ in the above equation. However, we will ignore the bias term for simplicity. Hence

$$\nabla_{\mathbf{w}} \log(\mu_n) = \frac{1}{\mu_n} \nabla_{\mathbf{w}} \mu_n = (1 - \mu_n) \mathbf{x}_n \quad (10.17)$$

Similarly,

$$\nabla_{\mathbf{w}} \log(1 - \mu_n) = \frac{-\mu_n(1 - \mu_n) \mathbf{x}_n}{1 - \mu_n} = -\mu_n \mathbf{x}_n \quad (10.18)$$

Thus the gradient vector of the NLL is given by

$$\nabla_{\mathbf{w}} \text{NLL}(\mathbf{w}) = -\frac{1}{N} \sum_{n=1}^N [y_n(1 - \mu_n) \mathbf{x}_n - (1 - y_n) \mu_n \mathbf{x}_n] \quad (10.19)$$

$$= -\frac{1}{N} \sum_{n=1}^N [y_n \mathbf{x}_n - y_n \mathbf{x}_n \mu_n - \mathbf{x}_n \mu_n + y_n \mathbf{x}_n \mu_n] \quad (10.20)$$

$$= \frac{1}{N} \sum_{n=1}^N (\mu_n - y_n) \mathbf{x}_n \quad (10.21)$$

If we interpret $e_n = \mu_n - y_n$ as an error signal, we can see that the gradient weights each input $\mathbf{x}_n$ by its error, and then averages the result. Note that we can rewrite the gradient in matrix form as follows:

$$\nabla_{\mathbf{w}} \text{NLL}(\mathbf{w}) = \frac{1}{N} (\mathbf{1}_N^\top (\text{diag}(\boldsymbol{\mu} - \mathbf{y}) \mathbf{X}))^\top \quad (10.22)$$

where $\mathbf{X}$ is the $N \times D$ design matrix containing the examples $\mathbf{x}_n$ in each row.

10.2.3.4 Deriving the Hessian

Gradient-based optimizers will find a stationary point where $\mathbf{g}(\mathbf{w}) = \mathbf{0}$. This could either be a global optimum or a local optimum. To be sure the stationary point is the global optimum, we must show that the objective is **convex**, for reasons we explain in Section 8.1.1.1. Intuitively this means that the NLL has a **bowl shape**, with a unique lowest point, which is indeed the case, as illustrated in Figure 10.5b.

More formally, we must prove that the Hessian is positive semi-definite, which we now do. (See Chapter 7 for relevant background information on linear algebra.) One can show that the Hessian is given by

$$\mathbf{H}(\mathbf{w}) = \nabla_{\mathbf{w}} \nabla_{\mathbf{w}}^\top \text{NLL}(\mathbf{w}) = \frac{1}{N} \sum_{n=1}^N (\mu_n(1 - \mu_n) \mathbf{x}_n) \mathbf{x}_n^\top = \frac{1}{N} \mathbf{X}^\top \mathbf{S} \mathbf{X} \quad (10.23)$$

where

$$\mathbf{S} \triangleq \text{diag}(\mu_1(1 - \mu_1), \dots, \mu_N(1 - \mu_N)) \quad (10.24)$$

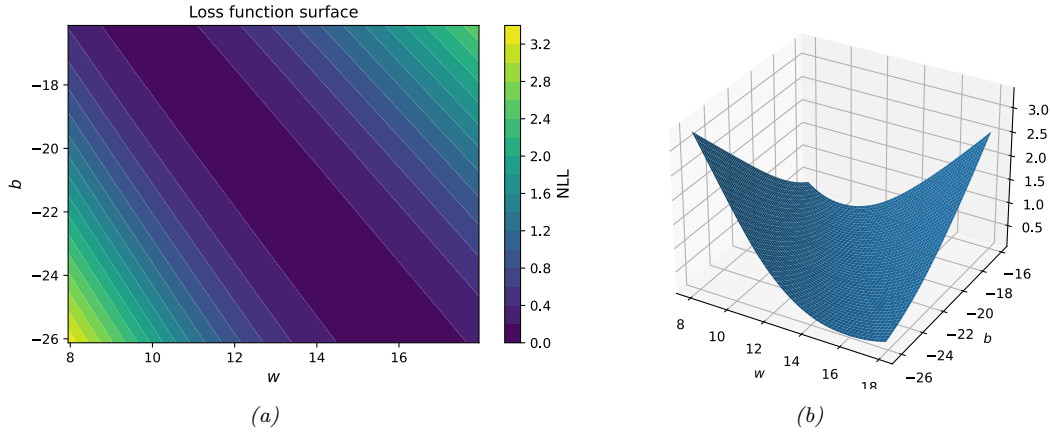


Figure 10.5: NLL loss surface for binary logistic regression applied to Iris dataset with 1 feature and 1 bias term. The goal is to minimize the function. The global MLE is at the center of the plot. Generated by [iris_logreg_loss_surface.ipynb](#).

We see that $\mathbf{H}$ is positive definite, since for any nonzero vector $\mathbf{v}$, we have

$$\mathbf{v}^\top \mathbf{X}^\top \mathbf{S} \mathbf{X} \mathbf{v} = (\mathbf{v}^\top \mathbf{X}^\top \mathbf{S}^{\frac{1}{2}})(\mathbf{S}^{\frac{1}{2}} \mathbf{X} \mathbf{v}) = \|\mathbf{v}^\top \mathbf{X}^\top \mathbf{S}^{\frac{1}{2}}\|_2^2 > 0 \quad (10.25)$$

This follows since $\mu_n > 0$ for all n , because of the use of the sigmoid function. Consequently the NLL is strictly convex. However, in practice, values of μ_n which are close to 0 or 1 might cause the Hessian to be close to singular. We can avoid this by using ℓ_2 regularization, as we discuss in Section 10.2.7.

10.2.4 Stochastic gradient descent

Our goal is to solve the following optimization problem

$$\hat{\mathbf{w}} \triangleq \underset{\mathbf{w}}{\operatorname{argmin}} \mathcal{L}(\mathbf{w}) \quad (10.26)$$

where $\mathcal{L}(\mathbf{w})$ is the loss function, in this case the negative log likelihood:

$$\text{NLL}(\mathbf{w}) = -\frac{1}{N} \sum_{n=1}^N [y_n \log \mu_n + (1 - y_n) \log(1 - \mu_n)] \quad (10.27)$$

where $\mu_n = \sigma(a_n)$ is the probability of class 1, and $a_n = \mathbf{w}^\top \mathbf{x}_n$ is the log odds.

There are many algorithms we could use to solve Equation (10.26), as we discuss in Chapter 8. Perhaps the simplest is to use stochastic gradient descent (Section 8.4). If we use a minibatch of size 1, then we get the following simple update equation:

$$\mathbf{w}_{t+1} = \mathbf{w}_t - \eta_t \nabla_{\mathbf{w}} \text{NLL}(\mathbf{w}_t) = \mathbf{w}_t - \eta_t (\mu_n - y_n) \mathbf{x}_n \quad (10.28)$$

where we replaced the average over all N examples in the gradient of Equation (10.21) with a single stochastically chosen sample n . (The index n changes with t .)

Since we know the objective is convex (see Section 10.2.3.4), then one can show that this procedure will converge to the global optimum, provided we decay the learning rate at the appropriate rate (see Section 8.4.3). We can improve the convergence speed using variance reduction techniques such as SAGA (Section 8.4.5.2).

10.2.5 Perceptron algorithm

A **perceptron**, first introduced in [Ros58], is a deterministic binary classifier of the following form:

$$f(\mathbf{x}_n; \boldsymbol{\theta}) = \mathbb{I}(\mathbf{w}^\top \mathbf{x}_n + b > 0) \quad (10.29)$$

This can be seen to be a limiting case of a binary logistic regression classifier, in which the sigmoid function $\sigma(a)$ is replaced by the Heaviside step function $H(a) \triangleq \mathbb{I}(a > 0)$. See Figure 2.10 for a comparison of these two functions.

Since the Heaviside function is not differentiable, we cannot use gradient-based optimization methods to fit this model. However, Rosenblatt proposed the **perceptron learning algorithm** instead. The basic idea is to start with random weights, and then iteratively update them whenever the model makes a prediction mistake. More precisely, we update the weights using

$$\mathbf{w}_{t+1} = \mathbf{w}_t - \eta_t(\hat{y}_n - y_n)\mathbf{x}_n \quad (10.30)$$

where $(\mathbf{x}_n, y_n)$ is the labeled example sampled at iteration t , and η_t is the learning rate or step size. (We can set the step size to 1, since the magnitude of the weights does not affect the decision boundary.) See [perceptron_demo_2d.ipynb](#) for a simple implementation of this algorithm.

The perceptron update rule in Equation (10.30) has an intuitive interpretation: if the prediction is correct, no change is made, otherwise we move the weights in a direction so as to make the correct answer more likely. More precisely, if $y_n = 1$ and $\hat{y}_n = 0$, we have $\mathbf{w}_{t+1} = \mathbf{w}_t + \mathbf{x}_n$, and if $y_n = 0$ and $\hat{y}_n = 1$, we have $\mathbf{w}_{t+1} = \mathbf{w}_t - \mathbf{x}_n$.

By comparing Equation (10.30) to Equation (10.28), we see that the perceptron update rule is equivalent to the SGD update rule for binary logistic regression using the approximation where we replace the soft probabilities $\mu_n = p(y_n = 1 | \mathbf{x}_n)$ with hard labels $\hat{y}_n = f(\mathbf{x}_n)$. The advantage of the perceptron method is that we don't need to compute probabilities, which can be useful when the label space is very large. The disadvantage is that the method will only converge when the data is linearly separable [Nov62], whereas SGD for minimizing the NLL for logistic regression will always converge to the globally optimal MLE, even if the data is not linearly separable.

In Section 13.2, we will generalize perceptrons to nonlinear functions, thus significantly enhancing their usefulness.

10.2.6 Iteratively reweighted least squares

Gradient descent is a **first order** optimization method, which means it only uses first order gradients to navigate through the loss landscape. This can be slow, especially when some directions of space point steeply downhill, whereas other have a shallower gradient, as is the case in Figure 10.5a. In such problems, it can be much faster to use a **second order** optimization method, that takes the curvature of the space into account.

We discuss such methods in more detail in Section 8.3. Here we just consider a simple second order method that works well for logistic regression. We focus on the full batch setting (so we assume N is small), since it is harder to make second order methods work in the stochastic setting (see e.g., [Byr+16; Liu+18b] for some methods).

The classic second-order method is **Newton's method**. This consists of updates of the form

$$\mathbf{w}_{t+1} = \mathbf{w}_t - \eta_t \mathbf{H}_t^{-1} \mathbf{g}_t \quad (10.31)$$

where

$$\mathbf{H}_t \triangleq \nabla^2 \mathcal{L}(\mathbf{w})|_{\mathbf{w}_t} = \nabla^2 \mathcal{L}(\mathbf{w}_t) = \mathbf{H}(\mathbf{w}_t) \quad (10.32)$$

is assumed to be positive-definite to ensure the update is well-defined. If the Hessian is exact, we can set the step size to $\eta_t = 1$.

We now apply this method to logistic regression. Recall from Section 10.2.3.3 that the gradient and Hessian are given by

$$\nabla_{\mathbf{w}} \text{NLL}(\mathbf{w}) = \frac{1}{N} \sum_{n=1}^N (\mu_n - y_n) \mathbf{x}_n \quad (10.33)$$

$$\mathbf{H} = \frac{1}{N} \mathbf{X}^\top \mathbf{S} \mathbf{X} \quad (10.34)$$

$$\mathbf{S} \triangleq \text{diag}(\mu_1(1 - \mu_1), \dots, \mu_N(1 - \mu_N)) \quad (10.35)$$

Hence the Newton update has the form

$$\mathbf{w}_{t+1} = \mathbf{w}_t - \mathbf{H}^{-1} \mathbf{g}_t \quad (10.36)$$

$$= \mathbf{w}_t + (\mathbf{X}^\top \mathbf{S}_t \mathbf{X})^{-1} \mathbf{X}^\top (\mathbf{y} - \boldsymbol{\mu}_t) \quad (10.37)$$

$$= (\mathbf{X}^\top \mathbf{S}_t \mathbf{X})^{-1} [(\mathbf{X}^\top \mathbf{S}_t \mathbf{X}) \mathbf{w}_t + \mathbf{X}^\top (\mathbf{y} - \boldsymbol{\mu}_t)] \quad (10.38)$$

$$= (\mathbf{X}^\top \mathbf{S}_t \mathbf{X})^{-1} \mathbf{X}^\top [\mathbf{S}_t \mathbf{X} \mathbf{w}_t + \mathbf{y} - \boldsymbol{\mu}_t] \quad (10.39)$$

$$= (\mathbf{X}^\top \mathbf{S}_t \mathbf{X})^{-1} \mathbf{X}^\top \mathbf{S}_t \mathbf{z}_t \quad (10.40)$$

where we have defined the **working response** as

$$\mathbf{z}_t \triangleq \mathbf{X} \mathbf{w}_t + \mathbf{S}_t^{-1} (\mathbf{y} - \boldsymbol{\mu}_t) \quad (10.41)$$

and $\mathbf{S}_t = \text{diag}(\mu_{t,n}(1 - \mu_{t,n}))$. Since $\mathbf{S}_t$ is a diagonal matrix, we can rewrite the targets in component form as follows:

$$z_{t,n} = \mathbf{w}_t^\top \mathbf{x}_n + \frac{y_n - \mu_{t,n}}{\mu_{t,n}(1 - \mu_{t,n})} \quad (10.42)$$

Equation (10.40) is an example of a weighted least squares problem (Section 11.2.2.4), which is a minimizer of

$$\sum_{n=1}^N S_{t,n} (z_{t,n} - \mathbf{w}_t^\top \mathbf{x}_n)^2 \quad (10.43)$$

Algorithm 10.1: Iteratively reweighted least squares (IRLS)

```

1  $\mathbf{w} = \mathbf{0}$ 
2 repeat
3   for  $n = 1 : N$  do
4      $a_n = \mathbf{w}^\top \mathbf{x}_n$ 
5      $\mu_n = \sigma(a_n)$ 
6      $s_n = \mu_n(1 - \mu_n)$ 
7      $z_n = a_n + \frac{y_n - \mu_n}{s_n}$ 
8    $\mathbf{S} = \text{diag}(s_{1:N})$ 
9    $\mathbf{w} = (\mathbf{X}^\top \mathbf{S} \mathbf{X})^{-1} \mathbf{X}^\top \mathbf{S} \mathbf{z}$ 
10 until converged

```

The overall method is therefore known as the **iteratively reweighted least squares (IRLS)** algorithm, since at each iteration we solve a weighted least squares problem, where the weight matrix $\mathbf{S}_t$ changes at each iteration. See Algorithm 10.1 for some pseudocode.

Note that **Fisher scoring** is the same as IRLS except we replace the Hessian of the actual log-likelihood with its expectation, i.e., we use the Fisher information matrix (Section 4.7.2) instead of $\mathbf{H}$. Since the Fisher information matrix is independent of the data, it can be precomputed, unlike the Hessian, which must be reevaluated at every iteration. This can be faster for problems with many parameters.

10.2.7 MAP estimation

In Figure 10.4, we saw how logistic regression can overfit when there are too many parameters compared to training examples. This is a consequence of the ability of maximum likelihood to find weights that force the decision boundary to “wobble” in just the right way so as to curve around the examples. To get this behavior, the weights often need to be set to large values. For example, in Figure 10.4, when we use degree $K = 1$, we find that the MLE for the two input weights (ignoring the bias) is

$$\hat{\mathbf{w}} = [0.51291712, 0.11866937] \quad (10.44)$$

When we use degree $K = 2$, we get

$$\hat{\mathbf{w}} = [2.27510513, 0.05970325, 11.84198867, 15.40355969, 2.51242311] \quad (10.45)$$

And when $K = 4$, we get

$$\hat{\mathbf{w}} = [-3.07813766, \dots, -59.03196044, 51.77152431, 10.25054164] \quad (10.46)$$

One way to reduce such overfitting is to prevent the weights from becoming so large. We can do this by using a zero-mean Gaussian prior, $p(\mathbf{w}) = \mathcal{N}(\mathbf{w}|\mathbf{0}, C\mathbf{I})$, and then using MAP estimation, as we discussed in Section 4.5.3. The new training objective becomes

$$\mathcal{L}(\mathbf{w}) = \text{NLL}(\mathbf{w}) + \lambda \|\mathbf{w}\|_2^2 \quad (10.47)$$

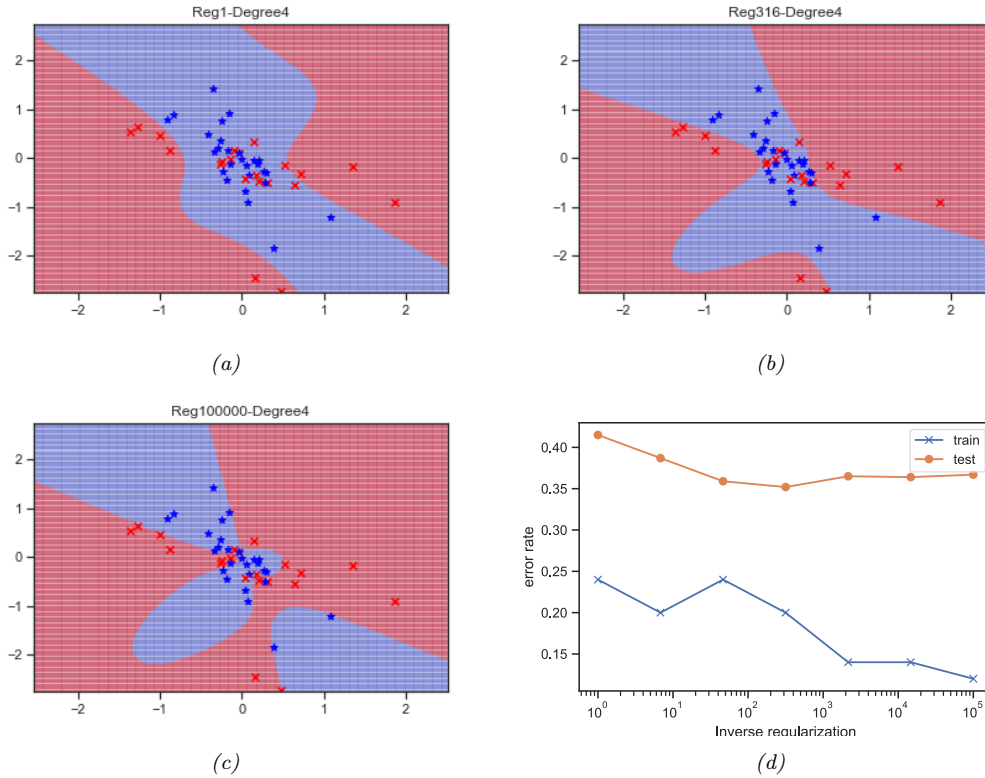


Figure 10.6: Weight decay with variance C applied to two-class, two-dimensional logistic regression problem with a degree 4 polynomial. (a) $C = 1$. (b) $C = 316$. (c) $C = 100,000$. (d) Train and test error vs C . Generated by [logreg_poly_demo.ipynb](#).

where $\|\mathbf{w}\|_2^2 = \sum_{d=1}^D w_d^2$ and $\lambda = 1/C$. This is called ℓ_2 **regularization** or **weight decay**. The larger the value of λ , the more the parameters are penalized for being “large” (deviating from the zero-mean prior), and thus the less flexible the model. See Figure 10.6 for an illustration.

We can compute the MAP estimate by slightly modifying the input to the above gradient-based optimization algorithms. The gradient and Hessian of the penalized negative log likelihood have the following forms:

$$\text{PNLL}(\mathbf{w}) = \text{NLL}(\mathbf{w}) + \lambda \mathbf{w}^\top \mathbf{w} \quad (10.48)$$

$$\nabla_{\mathbf{w}} \text{PNLL}(\mathbf{w}) = \mathbf{g}(\mathbf{w}) + 2\lambda \mathbf{w} \quad (10.49)$$

$$\nabla_{\mathbf{w}}^2 \text{PNLL}(\mathbf{w}) = \mathbf{H}(\mathbf{w}) + 2\lambda \mathbf{I} \quad (10.50)$$

where $\mathbf{g}(\mathbf{w})$ is the gradient and $\mathbf{H}(\mathbf{w})$ is the Hessian of the unpenalized NLL.

For an interesting exercise related to ℓ_2 regularized logistic regression, see Exercise 10.2.

10.2.8 Standardization

In Section 10.2.7, we use an isotropic prior $\mathcal{N}(\mathbf{w}|\mathbf{0}, \lambda^{-1}\mathbf{I})$ to prevent overfitting. This implicitly encodes the assumption that we expect all weights to be similar in magnitude, which in turn encodes the assumption we expect all input features to be similar in magnitude. However, in many datasets, input features are on different scales. In such cases, it is common to **standardize** the data, to ensure each feature has mean 0 and variance 1. We can do this by subtracting the mean and dividing by the standard deviation of each feature, as follows:

$$\text{standardize}(x_{nd}) = \frac{x_{nd} - \hat{\mu}_d}{\hat{\sigma}_d} \quad (10.51)$$

$$\hat{\mu}_d = \frac{1}{N} \sum_{n=1}^N x_{nd} \quad (10.52)$$

$$\hat{\sigma}_d^2 = \frac{1}{N} \sum_{n=1}^N (x_{nd} - \hat{\mu}_d)^2 \quad (10.53)$$

An alternative is to use **min-max scaling**, in which we rescale the inputs so they lie in the interval $[0, 1]$. Both methods ensure the features are comparable in magnitude, which can help with model fitting and inference, even if we don't use MAP estimation. (See Section 11.7.5 for a discussion of this point.)

10.3 Multinomial logistic regression

Multinomial logistic regression is a discriminative classification model of the following form:

$$p(y|\mathbf{x}, \boldsymbol{\theta}) = \text{Cat}(y|\text{softmax}(\mathbf{W}\mathbf{x} + \mathbf{b})) \quad (10.54)$$

where $\mathbf{x} \in \mathbb{R}^D$ is the input vector, $y \in \{1, \dots, C\}$ is the class label, $\text{softmax}()$ is the softmax function (Section 2.5.2), $\mathbf{W}$ is a $C \times D$ weight matrix, $\mathbf{b}$ is C -dimensional bias vector, $\boldsymbol{\theta} = (\mathbf{W}, \mathbf{b})$ are all the parameters. (We will henceforth assume we have prepended each $\mathbf{x}$ with a 1, and added $\mathbf{b}$ to the first column of $\mathbf{W}$, so this simplifies to $\boldsymbol{\theta} = \mathbf{W}$.)

If we let $\mathbf{a} = \mathbf{W}\mathbf{x}$ be the C -dimensional vector of **logits**, then we can rewrite the above as follows:

$$p(y = c|\mathbf{x}, \boldsymbol{\theta}) = \frac{e^{a_c}}{\sum_{c'=1}^C e^{a_{c'}}} \quad (10.55)$$

Because of the normalization condition $\sum_{c=1}^C p(y_n = c|\mathbf{x}_n, \boldsymbol{\theta}) = 1$, we can set $\mathbf{w}_C = \mathbf{0}$. (For example, in binary logistic regression, where $C = 2$, we only learn a single weight vector.) Therefore the parameters $\boldsymbol{\theta}$ correspond to a weight matrix $\mathbf{W}$ of size $(C - 1) \times D$, where $\mathbf{x}_n \in \mathbb{R}^D$.

Note that this model assumes the labels are mutually exclusive, i.e., there is only one true label. For some applications (e.g., **image tagging**), we want to predict one or more labels for an input; in this case, the output space is the set of subsets of $\{1, \dots, C\}$. This is called **multi-label classification**, as opposed to **multi-class classification**. This can be viewed as a bit vector, $\mathcal{Y} = \{0, 1\}^C$, where the c 'th output is set to 1 if the c 'th tag is present. We can tackle this using a modified version of

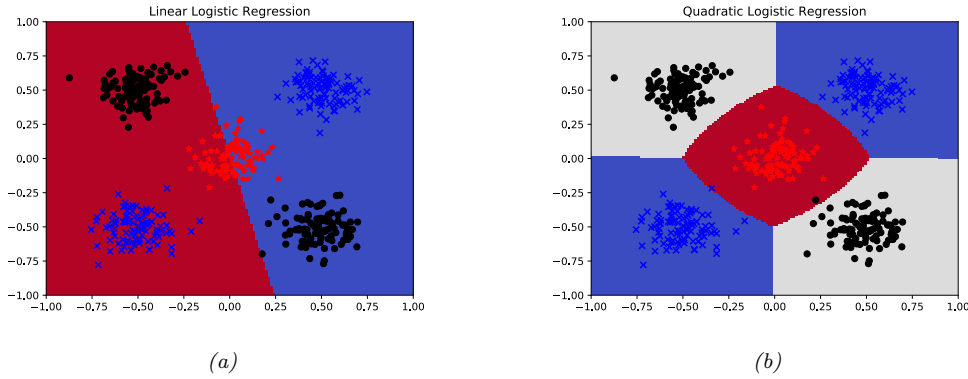


Figure 10.7: Example of 3-class logistic regression with 2d inputs. (a) Original features. (b) Quadratic features. Generated by [logreg_multiclass_demo.ipynb](#).

binary logistic regression with multiple outputs:

$$p(\mathbf{y}|\mathbf{x}, \boldsymbol{\theta}) = \prod_{c=1}^C \text{Ber}(y_c | \sigma(\mathbf{w}_c^\top \mathbf{x})) \quad (10.56)$$

10.3.1 Linear and nonlinear classifiers

Logistic regression computes linear decision boundaries in the input space, as shown in Figure 10.7(a) for the case where $\mathbf{x} \in \mathbb{R}^2$ and we have $C = 3$ classes. However, we can always transform the inputs in some way to create nonlinear boundaries. For example, suppose we replace $\mathbf{x} = (x_1, x_2)$ by

$$\boldsymbol{\phi}(\mathbf{x}) = [1, x_1, x_2, x_1^2, x_2^2, x_1 x_2] \quad (10.57)$$

This lets us create quadratic decision boundaries, as illustrated in Figure 10.7(b).

10.3.2 Maximum likelihood estimation

In this section, we discuss how to compute the maximum likelihood estimate (MLE) by minimizing the negative log likelihood (NLL).

10.3.2.1 Objective

The NLL is given by

$$\text{NLL}(\boldsymbol{\theta}) = -\frac{1}{N} \log \prod_{n=1}^N \prod_{c=1}^C \mu_{nc}^{y_{nc}} = -\frac{1}{N} \sum_{n=1}^N \sum_{c=1}^C y_{nc} \log \mu_{nc} = \frac{1}{N} \sum_{n=1}^N \mathbb{H}_{ce}(\mathbf{y}_n, \boldsymbol{\mu}_n) \quad (10.58)$$

where $\mu_{nc} = p(y_{nc} = 1 | \mathbf{x}_n, \boldsymbol{\theta}) = \text{softmax}(f(\mathbf{x}_n, \boldsymbol{\theta}))_c$, $\mathbf{y}_n$ is the one-hot encoding of the label (so $y_{nc} = \mathbb{I}(y_n = c)$), and $\mathbb{H}_{ce}(\mathbf{y}_n, \boldsymbol{\mu}_n)$ is the cross-entropy:

$$\mathbb{H}_{ce}(\mathbf{p}, \mathbf{q}) = - \sum_{c=1}^C p_c \log q_c \quad (10.59)$$

10.3.2.2 Optimizing the objective

To find the optimum, we need to solve $\nabla_{\mathbf{w}} \text{NLL}(\mathbf{w}) = \mathbf{0}$, where $\mathbf{w}$ is a vectorized version of the weight matrix $\mathbf{W}$, and where we are ignoring the bias term for notational simplicity. We can find such a stationary point using any gradient-based optimizer; we give some examples below. But first we derive the gradient and Hessian, and then prove that the objective is convex.

10.3.2.3 Deriving the gradient

To derive the gradient of the NLL, we need to use the Jacobian of the softmax function, which is as follows (see Exercise 10.1 for the proof):

$$\frac{\partial \mu_c}{\partial a_j} = \mu_c (\delta_{cj} - \mu_j) \quad (10.60)$$

where $\delta_{cj} = \mathbb{I}(c = j)$. For example, if we have 3 classes, the Jacobian matrix is given by

$$\left[\frac{\partial \mu_c}{\partial a_j} \right]_{cj} = \begin{pmatrix} \mu_1(1 - \mu_1) & -\mu_1\mu_2 & -\mu_1\mu_3 \\ -\mu_2\mu_1 & \mu_2(1 - \mu_2) & -\mu_2\mu_3 \\ -\mu_3\mu_1 & -\mu_3\mu_2 & \mu_3(1 - \mu_3) \end{pmatrix} \quad (10.61)$$

In matrix form, this can be written as

$$\frac{\partial \boldsymbol{\mu}}{\partial \mathbf{a}} = (\boldsymbol{\mu} \mathbf{1}^\top) \odot (\mathbf{I} - \mathbf{1} \boldsymbol{\mu}^\top) \quad (10.62)$$

where $\odot$ is elementwise product, $\boldsymbol{\mu} \mathbf{1}^\top$ copies $\boldsymbol{\mu}$ across each column, and $\mathbf{1} \boldsymbol{\mu}^\top$ copies $\boldsymbol{\mu}$ across each row.

We now derive the gradient of the NLL for a single example, indexed by n . To do this, we flatten the $D \times C$ weight matrix into a vector $\mathbf{w}$ of size CD (or $(C - 1)D$ if we freeze one of the classes to have zero weight) by concatenating the rows, and then transposing into a column vector. We use $\mathbf{w}_j$ to denote the vector of weights associated with class j . The gradient wrt this vector is giving by the

following (where we use the Kronecker delta notation, δ_{jc} , which equals 1 if $j = c$ and 0 otherwise):

$$\nabla_{\mathbf{w}_j} \text{NLL}_n = \sum_c \frac{\partial \text{NLL}_n}{\partial \mu_{nc}} \frac{\partial \mu_{nc}}{\partial a_{nj}} \frac{\partial a_{nj}}{\partial \mathbf{w}_j} \quad (10.63)$$

$$= - \sum_c \frac{y_{nc}}{\mu_{nc}} \mu_{nc} (\delta_{jc} - \mu_{nj}) \mathbf{x}_n \quad (10.64)$$

$$= \sum_c y_{nc} (\mu_{nj} - \delta_{jc}) \mathbf{x}_n \quad (10.65)$$

$$= \left(\sum_c y_{nc} \right) \mu_{nj} \mathbf{x}_n - \sum_c \delta_{jc} y_{nc} \mathbf{x}_n \quad (10.66)$$

$$= (\mu_{nj} - y_{nj}) \mathbf{x}_n \quad (10.67)$$

We can repeat this computation for each class, to get the full gradient vector. The gradient of the overall NLL is obtained by summing over examples, to give the $D \times C$ matrix

$$\mathbf{g}(\mathbf{w}) = \frac{1}{N} \sum_{n=1}^N \mathbf{x}_n (\boldsymbol{\mu}_n - \mathbf{y}_n)^\top \quad (10.68)$$

This has the same form as in the binary logistic regression case, namely an error term times the input.

10.3.2.4 Deriving the Hessian

Exercise 10.1 asks you to show that the Hessian of the NLL for multinomial logistic regression is given by

$$\mathbf{H}(\mathbf{w}) = \frac{1}{N} \sum_{n=1}^N (\text{diag}(\boldsymbol{\mu}_n) - \boldsymbol{\mu}_n \boldsymbol{\mu}_n^\top) \otimes (\mathbf{x}_n \mathbf{x}_n^\top) \quad (10.69)$$

where $\mathbf{A} \otimes \mathbf{B}$ is the Kronecker product (Section 7.2.5). In other words, the block c, c' submatrix is given by

$$\mathbf{H}_{c,c'}(\mathbf{w}) = \frac{1}{N} \sum_n \mu_{nc} (\delta_{c,c'} - \mu_{n,c'}) \mathbf{x}_n \mathbf{x}_n^\top \quad (10.70)$$

For example, if we have 3 features and 2 classes, this becomes

$$\mathbf{H}(\mathbf{w}) = \frac{1}{N} \sum_n \begin{pmatrix} \mu_{n1} - \mu_{n1}^2 & -\mu_{n1}\mu_{n2} \\ -\mu_{n1}\mu_{n2} & \mu_{n2} - \mu_{n2}^2 \end{pmatrix} \otimes \begin{pmatrix} x_{n1}x_{n1} & x_{n1}x_{n2} & x_{n1}x_{n3} \\ x_{n2}x_{n1} & x_{n2}x_{n2} & x_{n2}x_{n3} \\ x_{n3}x_{n1} & x_{n3}x_{n2} & x_{n3}x_{n3} \end{pmatrix} \quad (10.71)$$

$$= \frac{1}{N} \sum_n \begin{pmatrix} (\mu_{n1} - \mu_{n1}^2) \mathbf{X}_n & -\mu_{n1}\mu_{n2} \mathbf{X}_n \\ -\mu_{n1}\mu_{n2} \mathbf{X}_n & (\mu_{n2} - \mu_{n2}^2) \mathbf{X}_n \end{pmatrix} \quad (10.72)$$

where $\mathbf{X}_n = \mathbf{x}_n \mathbf{x}_n^\top$. Exercise 10.1 also asks you to show that this is a positive definite matrix, so the objective is convex.

10.3.3 Gradient-based optimization

It is straightforward to use the gradient in Section 10.3.2.3 to derive the SGD algorithm. Similarly, we can use the Hessian in Section 10.3.2.4 to derive a second-order optimization method. However, computing the Hessian can be expensive, so it is common to approximate it using quasi-Newton methods, such as limited memory BFGS. (BFGS stands for Broyden, Fletcher, Goldfarb and Shanno.) See Section 8.3.2 for details. Another approach, which is similar to IRLS, is described in Section 10.3.4.

All of these methods rely on computing the gradient of the log-likelihood, which in turn requires computing normalized probabilities, which can be computed from the logits vector $\mathbf{a} = \mathbf{W}\mathbf{x}$ using

$$p(y = c|\mathbf{x}) = \exp(a_c - \text{lse}(\mathbf{a})) \quad (10.73)$$

where lse is the log-sum-exp function defined in Section 2.5.4. For this reason, many software libraries define a version of the cross-entropy loss that takes unnormalized logits as input.

10.3.4 Bound optimization

In this section, we consider an approach for fitting logistic regression using a class of algorithms known as bound optimization, which we describe in Section 8.7. The basic idea is to iteratively construct a lower bound on the function you want to maximize, and then to update the bound, so it “pushes up” on the true function. Optimizing the bound is often easier than updating the function directly.

If $\ell(\boldsymbol{\theta})$ is a concave function we want to maximize, then one way to obtain a valid lower bound is to use a bound on its Hessian, i.e., to find a negative definite matrix $\mathbf{B}$ such that $\mathbf{H}(\boldsymbol{\theta}) \succ \mathbf{B}$. In this case, one can show that

$$\ell(\boldsymbol{\theta}) \geq \ell(\boldsymbol{\theta}^t) + (\boldsymbol{\theta} - \boldsymbol{\theta}^t)^\top \mathbf{g}(\boldsymbol{\theta}^t) + \frac{1}{2}(\boldsymbol{\theta} - \boldsymbol{\theta}^t)^\top \mathbf{B}(\boldsymbol{\theta} - \boldsymbol{\theta}^t) \quad (10.74)$$

where $\mathbf{g}(\boldsymbol{\theta}^t) = \nabla \ell(\boldsymbol{\theta}^t)$. Defining $Q(\boldsymbol{\theta}, \boldsymbol{\theta}^t)$ as the right-hand-side of Equation (10.74), the update becomes

$$\boldsymbol{\theta}^{t+1} = \boldsymbol{\theta}^t - \mathbf{B}^{-1} \mathbf{g}(\boldsymbol{\theta}^t) \quad (10.75)$$

This is similar to a Newton update, except we use $\mathbf{B}$, which is a fixed matrix, rather than $\mathbf{H}(\boldsymbol{\theta}^t)$, which changes at each iteration. This can give us some of the advantages of second order methods at lower computational cost.

Let us now apply this to logistic regression, following [Kri+05]. Let $\boldsymbol{\mu}_n(\mathbf{w}) = [p(y_n = 1|\mathbf{x}_n, \mathbf{w}), \dots, p(y_n = C|\mathbf{x}_n, \mathbf{w})]$ and $\mathbf{y}_n = [\mathbb{I}(y_n = 1), \dots, \mathbb{I}(y_n = C)]$. We want to *maximize* the log-likelihood, which is as follows:

$$\ell(\mathbf{w}) = \sum_{n=1}^N \left[\sum_{c=1}^C y_{nc} \mathbf{w}_c^\top \mathbf{x}_n - \log \sum_{c=1}^C \exp(\mathbf{w}_c^\top \mathbf{x}_n) \right] \quad (10.76)$$

The gradient is given by the following (see Section 10.3.2.3 for details of the derivation):

$$\mathbf{g}(\mathbf{w}) = \sum_{n=1}^N (\mathbf{y}_n - \boldsymbol{\mu}_n(\mathbf{w})) \otimes \mathbf{x}_n \quad (10.77)$$

where $\otimes$ denotes Kronecker product (which, in this case, is just outer product of the two vectors). The Hessian is given by the following (see Section 10.3.2.4 for details of the derivation):

$$\mathbf{H}(\mathbf{w}) = - \sum_{n=1}^N (\text{diag}(\boldsymbol{\mu}_n(\mathbf{w})) - \boldsymbol{\mu}_n(\mathbf{w})\boldsymbol{\mu}_n(\mathbf{w})^\top) \otimes (\mathbf{x}_n \mathbf{x}_n^\top) \quad (10.78)$$

We can construct a lower bound on the Hessian, as shown in [Boh92]:

$$\mathbf{H}(\mathbf{w}) \succ -\frac{1}{2}[\mathbf{I} - \mathbf{1}\mathbf{1}^\top/C] \otimes \left(\sum_{n=1}^N \mathbf{x}_n \mathbf{x}_n^\top \right) \triangleq \mathbf{B} \quad (10.79)$$

where $\mathbf{I}$ is a C -dimensional identity matrix, and $\mathbf{1}$ is a C -dimensional vector of all 1s.¹ In the binary case, this becomes

$$\mathbf{H}(\mathbf{w}) \succ -\frac{1}{2} \left(1 - \frac{1}{2} \right) \left(\sum_{n=1}^N \mathbf{x}_n \mathbf{x}_n^\top \right) = -\frac{1}{4} \mathbf{X}^\top \mathbf{X} \quad (10.80)$$

This follows since $\mu_n \leq 0.5$ so $-(\mu_n - \mu_n^2) \geq -0.25$.

We can use this lower bound to construct an MM algorithm to find the MLE. The update becomes

$$\mathbf{w}^{t+1} = \mathbf{w}^t - \mathbf{B}^{-1} \mathbf{g}(\mathbf{w}^t) \quad (10.81)$$

This iteration can be faster than IRLS (Section 10.2.6) since we can precompute $\mathbf{B}^{-1}$ in time independent of N , rather than having to invert the Hessian at each iteration. For example, let us consider the binary case, so $\mathbf{g}^t = \nabla \ell(\mathbf{w}^t) = \mathbf{X}^\top (\mathbf{y} - \boldsymbol{\mu}^t)$, where $\boldsymbol{\mu}^t = [p_n(\mathbf{w}^t), (1 - p_n(\mathbf{w}^t))]_{n=1}^N$. The update becomes

$$\mathbf{w}^{t+1} = \mathbf{w}^t - 4(\mathbf{X}^\top \mathbf{X})^{-1} \mathbf{g}^t \quad (10.82)$$

Compare this to Equation (10.37), which has the following form:

$$\mathbf{w}^{t+1} = \mathbf{w}^t - \mathbf{H}^{-1} \mathbf{g}(\mathbf{w}^t) = \mathbf{w}^t - (\mathbf{X}^\top \mathbf{S}^t \mathbf{X})^{-1} \mathbf{g}^t \quad (10.83)$$

where $\mathbf{S}^t = \text{diag}(\boldsymbol{\mu}^t \odot (1 - \boldsymbol{\mu}^t))$. We see that Equation (10.82) is faster to compute, since we can precompute the constant matrix $(\mathbf{X}^\top \mathbf{X})^{-1}$.

10.3.5 MAP estimation

In Section 10.2.7 we discussed the benefits of ℓ_2 regularization for binary logistic regression. These benefits hold also in the multi-class case. However, there is also an additional, and surprising, benefit to do with **identifiability** of the parameters, as pointed out in [HTF09, Ex.18.3]. (We say that the parameters are identifiable if there is a unique value that maximizes the likelihood; equivalently, we require that the NLL be *strictly* convex.)

1. If we enforce that $\mathbf{w}_C = \mathbf{0}$, we can use $C - 1$ dimensions for these vectors / matrices.

To see why identifiability is an issue, recall that multiclass logistic regression has the form

$$p(y = c|\mathbf{x}, \mathbf{W}) = \frac{\exp(\mathbf{w}_c^T \mathbf{x})}{\sum_{k=1}^C \exp(\mathbf{w}_k^T \mathbf{x})} \quad (10.84)$$

where $\mathbf{W}$ is a $C \times D$ weight matrix. We can arbitrarily define $\mathbf{w}_c = \mathbf{0}$ for one of the classes, say $c = C$, since $p(y = C|\mathbf{x}, \mathbf{W}) = 1 - \sum_{c=1}^{C-1} p(y = c|\mathbf{x}, \mathbf{w})$. In this case, the model has the form

$$p(y = c|\mathbf{x}, \mathbf{W}) = \frac{\exp(\mathbf{w}_c^T \mathbf{x})}{1 + \sum_{k=1}^{C-1} \exp(\mathbf{w}_k^T \mathbf{x})} \quad (10.85)$$

If we don't "clamp" one of the vectors to some constant value, the parameters will be unidentifiable.

However, suppose we don't clamp $\mathbf{w}_c = \mathbf{0}$, so we are using Equation 10.84, but we add ℓ_2 regularization by optimizing

$$\text{PNLL}(\mathbf{W}) = - \sum_{n=1}^N \log p(y_n|\mathbf{x}_n, \mathbf{W}) + \lambda \sum_{c=1}^C \|\mathbf{w}_c\|_2^2 \quad (10.86)$$

where we have absorbed the $1/N$ term into λ . At the optimum we have $\sum_{c=1}^C \hat{w}_{cj} = 0$ for $j = 1 : D$, so the weights automatically satisfy a sum-to-zero constraint, thus making them uniquely identifiable.

To see why, note that at the optimum we have

$$\nabla \text{NLL}(\mathbf{w}) + 2\lambda \mathbf{w} = \mathbf{0} \quad (10.87)$$

$$\sum_n (\mathbf{y}_n - \boldsymbol{\mu}_n) \otimes \mathbf{x}_n = \lambda \mathbf{w} \quad (10.88)$$

Hence for any feature dimension j we have

$$\lambda \sum_c w_{cj} = \sum_n \sum_c (y_{nc} - \mu_{nc}) x_{nj} = \sum_n (\sum_c y_{nc} - \sum_c \mu_{nc}) x_{nj} = \sum_n (1 - 1) x_{nj} = 0 \quad (10.89)$$

Thus if $\lambda > 0$ we have $\sum_c \hat{w}_{cj} = 0$, so the weights will sum to zero across classes for each feature dimension.

10.3.6 Maximum entropy classifiers

Recall that the multinomial logistic regression model can be written as

$$p(y = c|\mathbf{x}, \mathbf{W}) = \frac{\exp(\mathbf{w}_c^T \mathbf{x})}{Z(\mathbf{w}, \mathbf{x})} = \frac{\exp(\mathbf{w}_c^T \mathbf{x})}{\sum_{c'=1}^C \exp(\mathbf{w}_{c'}^T \mathbf{x})} \quad (10.90)$$

where $Z(\mathbf{w}, \mathbf{x}) = \sum_c \exp(\mathbf{w}_c^T \mathbf{x})$ is the partition function (normalization constant). This uses the same features, but a different weight vector, for every class. There is a slight extension of this model that allows us to use features that are class-dependent. This model can be written as

$$p(y = c|\mathbf{x}, \mathbf{w}) = \frac{1}{Z(\mathbf{w}, \mathbf{x})} \exp(\mathbf{w}^T \boldsymbol{\phi}(\mathbf{x}, c)) \quad (10.91)$$

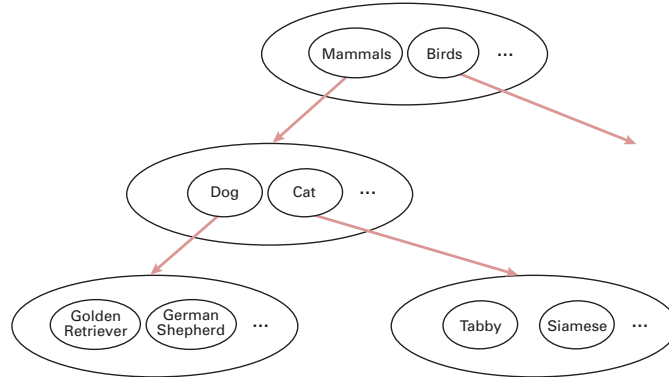


Figure 10.8: A simple example of a label hierarchy. Nodes within the same ellipse have a mutual exclusion relationship between them.

where $\phi(\mathbf{x}, c)$ is the feature vector for class c . This is called a **maximum entropy classifier**, or **maxent classifier** for short. (The origin of this term is explained in Section 3.4.4.)

Maxent classifiers include multinomial logistic regression as a special case. To see this let $\mathbf{w} = [\mathbf{w}_1, \dots, \mathbf{w}_C]$, and define the feature vector as follows:

$$\phi(\mathbf{x}, c) = [\mathbf{0}, \dots, \mathbf{x}, \dots, \mathbf{0}] \quad (10.92)$$

where $\mathbf{x}$ is embedded in the c 'th block, and the remaining blocks are zero. In this case, $\mathbf{w}^T \phi(\mathbf{x}, c) = \mathbf{w}_c^T \mathbf{x}$, so we recover multinomial logistic regression.

Maxent classifiers are very widely used in the field of natural language processing. For example, consider the problem of **semantic role labeling**, where we classify a word $\mathbf{x}$ into a semantic role y , such as person, place or thing. We might define (binary) features such as the following:

$$\phi_1(\mathbf{x}, y) = \mathbb{I}(y = \text{person} \wedge \mathbf{x} \text{ occurs after "Mr." or "Mrs"}) \quad (10.93)$$

$$\phi_2(\mathbf{x}, y) = \mathbb{I}(y = \text{person} \wedge \mathbf{x} \text{ is in whitelist of common names}) \quad (10.94)$$

$$\phi_3(\mathbf{x}, y) = \mathbb{I}(y = \text{place} \wedge \mathbf{x} \text{ is in Google maps}) \quad (10.95)$$

$\vdots$

We see that the features we use depend on the label.

There are two main ways of creating these features. The first is to manually specify many possibly useful features using various templates, and then use a feature selection algorithm, such as the group lasso method of Section 11.4.7. The second is to incrementally add features to the model, using a heuristic feature generation method.

10.3.7 Hierarchical classification

Sometimes the set of possible labels can be structured into a **hierarchy** or **taxonomy**. For example, we might want to predict what kind of an animal is in an image: it could be a dog or a cat; if it is a

dog, it could be a golden retriever or a German shepherd, etc. Intuitively, it makes sense to try to predict the most precise label for which we are confident [Den+12], that is, the system should “hedge its bets”.

One simple way to achieve this, proposed in [RF17], is as follows. First, create a model with a binary output label for every possible node in the tree. Before training the model, we will use **label smearing**, so that a label is propagated to all of its parents (**hypernyms**). For example, if an image is labeled “golden retriever”, we will also label it “dog”. If we train a multi-label classifier (which produces a vector $p(\mathbf{y}|\mathbf{x})$ of binary labels) on such smeared data, it will perform hierarchical classification, predicting a set of labels at different levels of abstraction.

However, this method could predict “golden retriever”, “cat” and “bird” all with probability 1.0, since the model does not capture the fact that some labels are mutually exclusive. To prevent this, we can add a mutual exclusion constraint between all label nodes which are siblings, as shown in Figure 10.8. For example, this model enforces that $p(\text{mammal}|\mathbf{x}) + p(\text{bird}|\mathbf{x}) = 1$, since these two labels are children of the root node. We can further partition the mammal probability into dogs and cats, so we have $p(\text{dog}|\mathbf{x}) + p(\text{cat}|\mathbf{x}) = p(\text{mammal}|\mathbf{x})$.

[Den+14; Din+15] generalize the above method by using a conditional graphical model where the graph structure can be more complex than a tree. In addition, they allow for soft constraints between labels, in addition to hard constraints.

10.3.8 Handling large numbers of classes

In this section, we discuss some issues that arise when there are a large number of potential labels, e.g., if the labels correspond to words from a language.

10.3.8.1 Hierarchical softmax

In regular softmax classifiers, computing the normalization constant, which is needed to compute the gradient of the log likelihood, takes $O(C)$ time, which can become the bottleneck if C is large. However, if we structure the labels as a tree, we can compute the probability of any label in $O(\log C)$ time, by multiplying the probabilities of each edge on the path from the root to the leaf. For example, consider the tree in Figure 10.9. We have

$$p(y = \text{Im}|C) = 0.57 \times 0.68 \times 0.72 = 0.28 \quad (10.96)$$

Thus we replace the “flat” output softmax with a tree-structured sequence of binary classifiers. This is called **hierarchical softmax** [Goo01; MB05].

A good way to structure such a tree is to use Huffman encoding, where the most frequent labels are placed near the top of the tree, as suggested in [Mik+13a]. (For a different approach, based on clustering the most common labels together, see [Gra+17]. And for yet another approach, based on sampling labels, see [Tit16].)

10.3.8.2 Class imbalance and the long tail

Another issue that often arises when there are a large number of classes is that for most classes, we may have very few examples. More precisely, if N_c is the number of examples of class c , then the empirical distribution $p(N_1, \dots, N_C)$ may have a **long tail**. The result is an extreme form of **class**

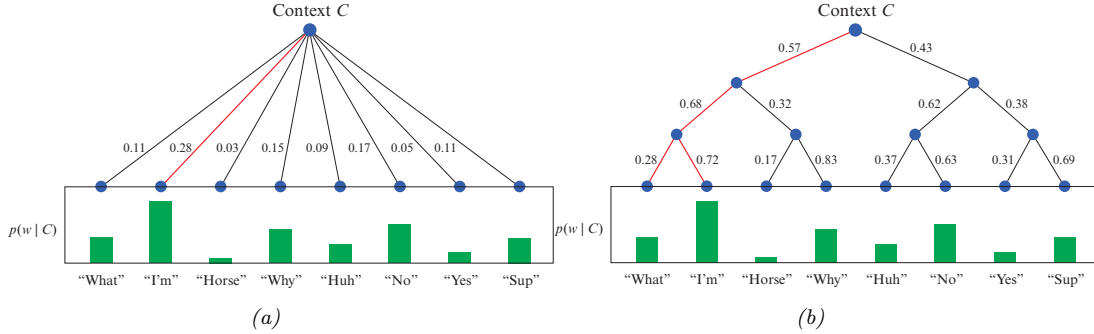


Figure 10.9: A flat and hierarchical softmax model $p(w|C)$, where C are the input features (context) and w is the output label (word). Adapted from <https://www.quora.com/What-is-hierarchical-softmax>.

imbalance (see e.g., [ASR15]). Since the rare classes will have a smaller effect on the overall loss than the common classes, the model may “focus its attention” on the common classes.

One method that can help is to set the bias terms $\mathbf{b}$ such that $\text{softmax}(\mathbf{b})_c = N_c/N$; such a model will match the empirical label prior even when using weights of $\mathbf{w} = \mathbf{0}$. We can then “subtract off” the prior term by using **logit adjustment** [Men+21], which ensures good performance across all groups.

Another common approach is to resample the data to make it more balanced, before (or during) training. In particular, suppose we sample a datapoint from class c with probability

$$p_c = \frac{N_c^q}{\sum_i^C N_i^q} \quad (10.97)$$

If we set $q = 1$, we recover standard **instance-balanced sampling**, where $p_c \propto N_c$; the common classes will be sampled more than rare classes. If we set $q = 0$, we recover **class-balanced sampling**, where $p_c = 1/C$; this can be thought of as first sampling a class uniformly at random, and then sampling an instance of this class. Finally, we can consider other options, such as $q = 0.5$, which is known as **square-root sampling** [Mah+18].

Yet another method that is simple and can easily handle the long tail is to use the **nearest class mean classifier**. This has the form

$$f(\mathbf{x}) = \underset{c}{\operatorname{argmin}} \|\mathbf{x} - \boldsymbol{\mu}_c\|_2^2 \quad (10.98)$$

where $\boldsymbol{\mu}_c = \frac{1}{N_c} \sum_{n:y_n=c} \mathbf{x}_n$ is the mean of the features belonging to class c . This induces a softmax posterior, as we discussed in Section 9.2.5. We can get much better results if we first use a neural network (see Part III) to learn good features, by training a DNN classifier with cross-entropy loss on the original unbalanced data. We then replace $\mathbf{x}$ with $\phi(\mathbf{x})$ in Equation (10.98). This simple approach can give very good performance on long-tailed distributions [Kan+20].

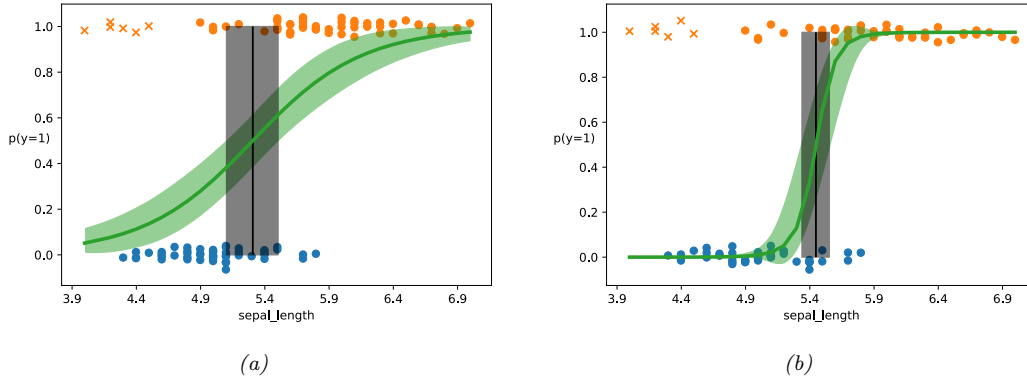


Figure 10.10: (a) Logistic regression on some data with outliers (denoted by x). Training points have been (vertically) jittered to avoid overlapping too much. Vertical line is the decision boundary, and its posterior credible interval. (b) Same as (a) but using robust model, with a mixture likelihood. Adapted from Figure 4.13 of [Mar18]. Generated by `logreg_iris_bayes_robust_1d_pymc3.ipynb`.

10.4 Robust logistic regression *

Sometimes we have **outliers** in our data, which are often due to labeling errors, also called **label noise**. To prevent the model from being adversely affected by such contamination, we will use **robust logistic regression**. In this section, we discuss some approaches to this problem. (Note that the methods can also be applied to DNNs. For a more thorough survey of label noise, and how it impacts deep learning, see [Han+20].)

10.4.1 Mixture model for the likelihood

One of the simplest ways to define a robust logistic regression model is to modify the likelihood so that it predicts that each output label y is generated uniformly at random with probability π , and otherwise is generated using the usual conditional model. In the binary case, this becomes

$$p(y|\mathbf{x}) = \pi \text{Ber}(y|0.5) + (1 - \pi) \text{Ber}(y|\sigma(\mathbf{w}^\top \mathbf{x})) \quad (10.99)$$

This approach, of using a mixture model for the observation model to make it robust, can be applied to many different models (e.g., DNNs).

We can fit this model using standard methods, such as SGD or Bayesian inference methods such as MCMC. For example, let us create a “contaminated” version of the 1d, two-class Iris dataset that we discussed in Section 4.6.7.2. We will add 6 examples of class 1 (Versicolor) with abnormally low sepal length. In Figure 10.10a, we show the results of fitting a standard (Bayesian) logistic regression model to this dataset. In Figure 10.10b, we show the results of fitting the above robust model. In the latter case, we see that the decision boundary is similar to the one we inferred from non-contaminated data, as shown in Figure 4.20b. We also see that the posterior uncertainty about the decision boundary’s location is smaller than when using a non-robust model.

10.4.2 Bi-tempered loss

In this section, we present an approach to robust logistic regression proposed in [Ami+19].

The first observation is that examples that are far from the decision boundary, but mislabeled, will have undue adverse affect on the model if the loss function is convex [LS10]. This can be overcome by replacing the usual cross entropy loss with a “tempered” version, that uses a temperature parameter $0 \leq t_1 < 1$ to ensure the loss from outliers is bounded. In particular, consider the standard relative entropy loss function:

$$\mathcal{L}(\mathbf{y}, \hat{\mathbf{y}}) = \mathbb{H}_{ce}(\mathbf{y}, \hat{\mathbf{y}}) = \sum_c y_c \log \hat{y}_c \quad (10.100)$$

where $\mathbf{y}$ is the true label distribution (often one-hot) and $\hat{\mathbf{y}}$ is the predicted distribution. We define the **tempered cross entropy** loss as follows:

$$\mathcal{L}(\mathbf{y}, \hat{\mathbf{y}}) = \sum_c \left[y_c (\log_{t_1} y_c - \log_{t_1} \hat{y}_c) - \frac{1}{2-t_1} (y_c^{2-t_1} - \hat{y}_c^{2-t_1}) \right] \quad (10.101)$$

which simplifies to the following when the true distribution $\mathbf{y}$ is one-hot, with all its mass on class c :

$$\mathcal{L}(c, \hat{\mathbf{y}}) = -\log_{t_1} \hat{y}_c - \frac{1}{2-t_1} \left(1 - \sum_{c'=1}^C \hat{y}_{c'}^{2-t_1} \right) \quad (10.102)$$

Here $\log_t$ is tempered version of the log function:

$$\log_t(x) \triangleq \frac{1}{1-t} (x^{1-t} - 1) \quad (10.103)$$

This is monotonically increasing and concave, and reduces to the standard (natural) logarithm when $t = 1$. (Similarly, tempered cross entropy reduces to standard cross entropy when $t = 1$.) However, the tempered log function is bounded from below by $-1/(1-t)$ for $0 \leq t < 1$, and hence the cross entropy loss is bounded from above (see Figure 10.11).

The second observation is that examples that are near the decision boundary, but mislabeled, need to use a transfer function (that maps from activations $\mathbb{R}^C$ to probabilities $[0, 1]^C$) that has heavier tails than the softmax, which is based on the exponential, so it can “look past” the neighborhood of the immediate examples. In particular, the standard softmax is defined by

$$\hat{y}_c = \frac{a_c}{\sum_{c'=1}^C \exp(a_{c'})} = \exp \left[a_c - \log \sum_{c'=1}^C \exp(a_{c'}) \right] \quad (10.104)$$

where $\mathbf{a}$ is the logits vector. We can make a heavy tailed version by using the **tempered softmax**, which uses a temperature parameter $t_2 > 1 > t_1$ as follows:

$$\hat{y}_c = \exp_{t_2}(a_c - \lambda_{t_2}(\mathbf{a})) \quad (10.105)$$

where

$$\exp_t(x) \triangleq [1 + (1-t)x]_+^{1/(1-t)} \quad (10.106)$$

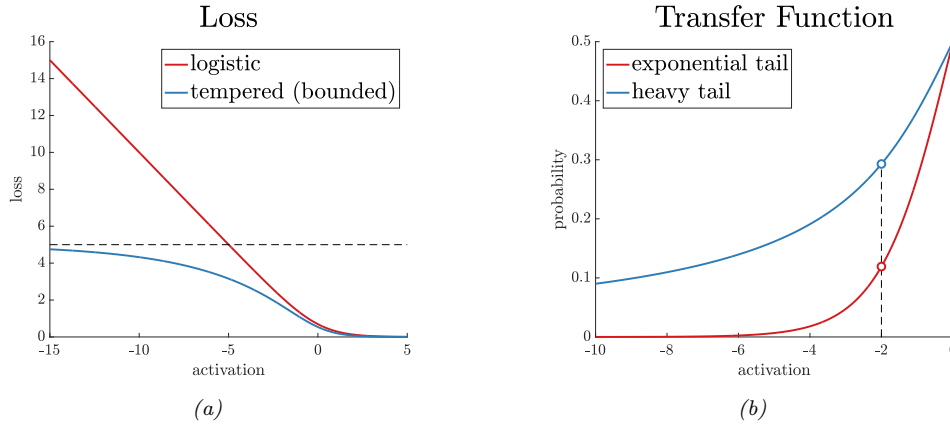


Figure 10.11: (a) Illustration of logistic and tempered logistic loss with $t_1 = 0.8$. (b) Illustration of sigmoid and tempered sigmoid transfer function with $t_2 = 2.0$. From <https://ai.googleblog.com/2019/08/bi-tempered-logistic-loss-for-training.html>. Used with kind permission of Ehsan Amid.

is a tempered version of the exponential function. (This reduces to the standard exponential function as $t \rightarrow 1$.) In Figure 10.11(right), we show that the tempered softmax (in the two-class case) has heavier tails, as desired.

All that remains is a way to compute $\lambda_{t_2}(\mathbf{a})$. This must satisfy the following fixed point equation:

$$\sum_{c=1}^C \exp_{t_2}(a_c - \lambda(\mathbf{a})) = 1 \quad (10.107)$$

We can solve for λ using binary search, or by using the iterative procedure in Algorithm 10.2.

Algorithm 10.2: Iterative algorithm for computing $\lambda(\mathbf{a})$ in Equation (10.107). From [AWS19].

```

1 Input: logits  $\mathbf{a}$ , temperature  $t > 1$ 
2  $\mu := \max(\mathbf{a})$ 
3  $\mathbf{a} := \mathbf{a} - \mu$ 
4 while  $\mathbf{a}$  not converged do
5    $Z(\mathbf{a}) := \sum_{c=1}^C \exp_t(a_c)$ 
6    $\mathbf{a} := Z(\mathbf{a})^{1-t}(\mathbf{a} - \mu \mathbf{1})$ 
7 Return  $-\log_t \frac{1}{Z(\mathbf{a})} + \mu$ 
```

Combining the tempered softmax with the tempered cross entropy results in a method called **bi-tempered logistic regression**. In Figure 10.12, we show an example of this in 2d. The top row is standard logistic regression, the bottom row is bi-tempered. The first column is clean data. The second column has label noise near the boundary. The robust version uses $t_1 = 1$ (standard cross entropy) but $t_2 = 4$ (tempered softmax with heavy tails). The third column has label noise far

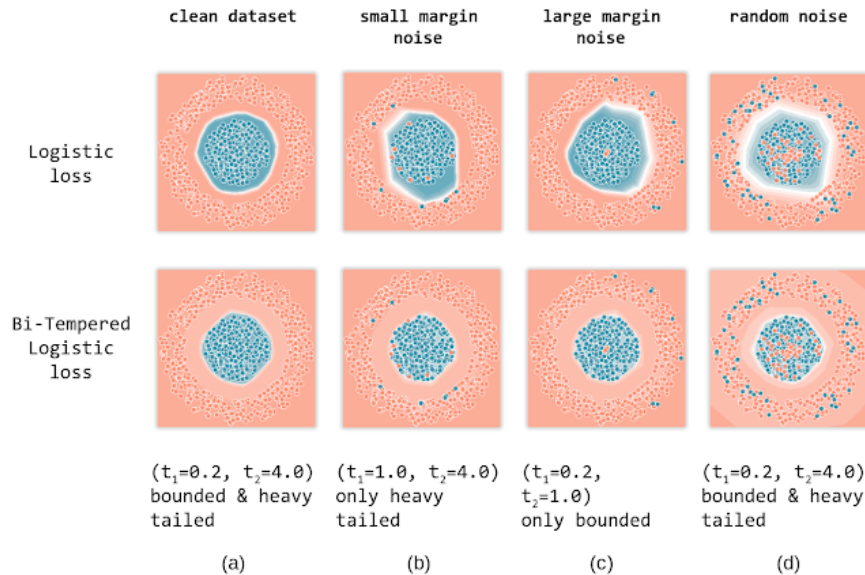


Figure 10.12: Illustration of standard and bi-tempered logistic regression on data with label noise. From <https://ai.googleblog.com/2019/08/bi-tempered-logistic-loss-for-training.html>. Used with kind permission of Ehsan Amid.

from the boundary. The robust version uses $t_1 = 0.2$ (tempered cross entropy with bounded loss) but $t_2 = 1$ (standard softmax). The fourth column has both kinds of noise; in this case, the robust version uses $t_1 = 0.2$ and $t_2 = 4$.

10.5 Bayesian logistic regression *

So far we have focused on point estimates of the parameters, either the MLE or the MAP estimate. However, in some cases we want to compute the posterior, $p(\mathbf{w}|\mathcal{D})$, in order to capture our uncertainty. This can be particularly useful in settings where we have little data, and where choosing the wrong decision may be costly.

Unlike with linear regression, it is not possible to compute the posterior exactly for a logistic regression model. A wide range of approximate algorithms can be used. In this section, we use one of the simplest, known as the Laplace approximation (Section 4.6.8.2). See the sequel to this book, [Mur23] for more advanced approximations.

10.5.1 Laplace approximation

As we discuss in Section 4.6.8.2, the Laplace approximation approximates the posterior using a Gaussian. The mean of the Gaussian is equal to the MAP estimate $\hat{\mathbf{w}}$, and the covariance is equal to the inverse Hessian $\mathbf{H}$ computed at the MAP estimate, i.e., $p(\mathbf{w}|\mathcal{D}) \approx \mathcal{N}(\mathbf{w}|\hat{\mathbf{w}}, \mathbf{H}^{-1})$. We can find the mode using a standard optimization method (see Section 10.2.7), and then we can use the results

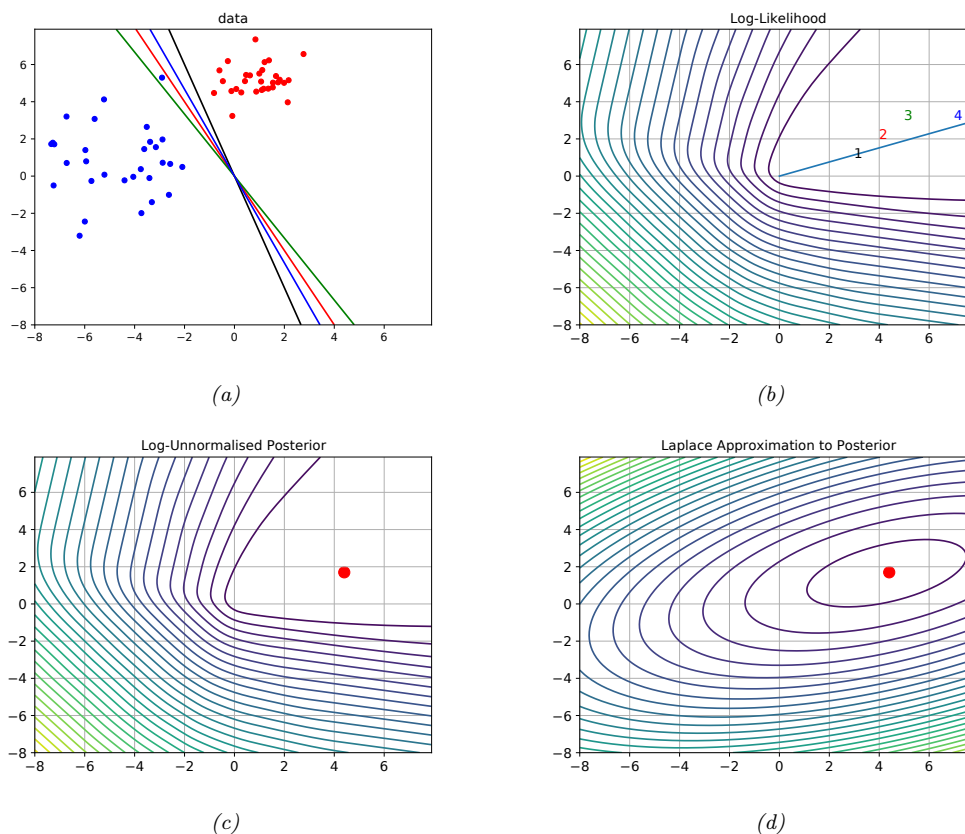


Figure 10.13: (a) Illustration of the data. (b) Log-likelihood for a logistic regression model. The line is drawn from the origin in the direction of the MLE (which is at infinity). The numbers correspond to 4 points in parameter space, corresponding to the lines in (a). (c) Unnormalized log posterior (assuming vague spherical prior). (d) Laplace approximation to posterior. Adapted from a figure by Mark Girolami. Generated by [logreg_laplace_demo.ipynb](#).

from Section 10.2.3.4 to compute the Hessian at the mode.

As an example, consider the data illustrated in Figure 10.13(a). There are many parameter settings that correspond to lines that perfectly separate the training data; we show 4 example lines. The likelihood surface is shown in Figure 10.13(b). The diagonal line connects the origin to the point in the grid with maximum likelihood, $\hat{\mathbf{w}}_{\text{mle}} = (8.0, 3.4)$. (The unconstrained MLE has $\|\mathbf{w}\| = \infty$, as we discussed in Section 10.2.7; this point can be obtained by following the diagonal line infinitely far to the right.)

For each decision boundary in Figure 10.13(a), we plot the corresponding parameter vector in Figure 10.13(b). These parameters values are $\mathbf{w}_1 = (3, 1)$, $\mathbf{w}_2 = (4, 2)$, $\mathbf{w}_3 = (5, 3)$, and $\mathbf{w}_4 = (7, 3)$. These points all approximately satisfy $\mathbf{w}_i(1)/\mathbf{w}_i(2) \approx \hat{\mathbf{w}}_{\text{mle}}(1)/\hat{\mathbf{w}}_{\text{mle}}(2)$, and hence are close to the

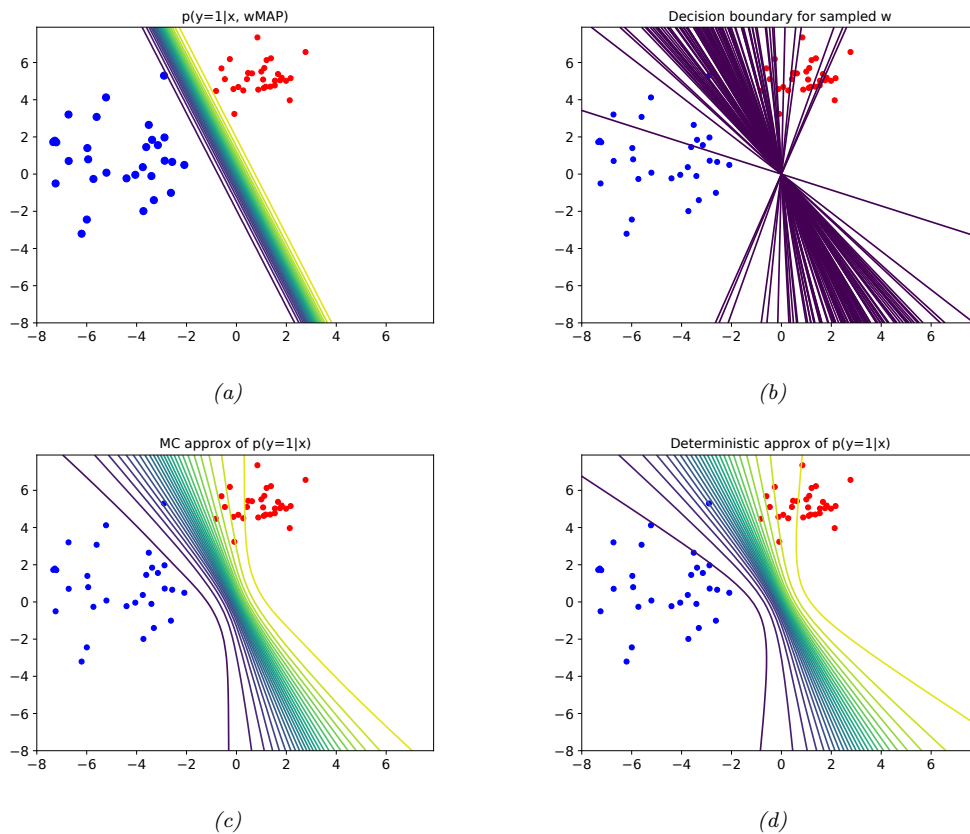


Figure 10.14: Posterior predictive distribution for a logistic regression model in 2d. (a): contours of $p(y=1|\mathbf{x}, \hat{\mathbf{w}}_{\text{MAP}})$. (b): samples from the posterior predictive distribution. (c): Averaging over these samples. (d): moderated output (probit approximation). Adapted from a figure by Mark Girolami. Generated by [logreg_laplace_demo.ipynb](#).

orientation of the maximum likelihood decision boundary. The points are ordered by increasing weight norm (3.16, 4.47, 5.83, and 7.62).

To ensure a unique solution, we use a (spherical) Gaussian prior centered at the origin, $\mathcal{N}(\mathbf{w}|\mathbf{0}, \sigma^2 \mathbf{I})$. The value of σ^2 controls the strength of the prior. If we set $\sigma^2 = 0$, we force the MAP estimate to be $\mathbf{w} = \mathbf{0}$; this will result in maximally uncertain predictions, since all points $\mathbf{x}$ will produce a predictive distribution of the form $p(y=1|\mathbf{x}) = 0.5$. If we set $\sigma^2 = \infty$, the prior becomes uninformative, and MAP estimate becomes the MLE, resulting in minimally uncertain predictions. (In particular, all positively labeled points will have $p(y=1|\mathbf{x}) = 1.0$, and all negatively labeled points will have $p(y=1|\mathbf{x}) = 0.0$, since the data is separable.) As a compromise (to make a nice illustration), we pick the value $\sigma^2 = 100$.

Multiplying this prior by the likelihood results in the unnormalized posterior shown in Fig-

ure 10.13(c). The MAP estimate is shown by the red dot. The Laplace approximation to this posterior is shown in Figure 10.13(d). We see that it gets the mode correct (by construction), but the shape of the posterior is somewhat distorted. (The southwest-northeast orientation captures uncertainty about the magnitude of $\mathbf{w}$, and the southeast-northwest orientation captures uncertainty about the orientation of the decision boundary.)

In Figure 10.14, we show contours of the posterior predictive distribution. Figure 10.14(a) shows the plugin approximation using the MAP estimate. We see that there is no uncertainty about the decision boundary, even though we are generating probabilistic predictions over the labels. Figure 10.14(b) shows what happens when we plug in samples from the Gaussian posterior. Now we see that there is considerable uncertainty about the orientation of the “best” decision boundary. Figure 10.14(c) shows the average of these samples. By averaging over multiple predictions, we see that the uncertainty in the decision boundary “splays out” as we move further from the training data. Figure 10.14(d) shows that the probit approximation gives very similar results to the Monte Carlo approximation.

10.5.2 Approximating the posterior predictive

The posterior $p(\mathbf{w}|\mathcal{D})$ tells us everything we know about the parameters of the model given the data. However, in machine learning applications, the main task of interest is usually to predict an output y given an input $\mathbf{x}$, rather than to try to understand the parameters of our model. Thus we need to compute the **posterior predictive distribution**

$$p(y|\mathbf{x}, \mathcal{D}) = \int p(y|\mathbf{x}, \mathbf{w})p(\mathbf{w}|\mathcal{D})d\mathbf{w} \quad (10.108)$$

As we discussed in Section 4.6.7.1, a simple approach to this is to first compute a point estimate $\hat{\mathbf{w}}$ of the parameters, such as the MLE or MAP estimate, and then to ignore all posterior uncertainty, by assuming $p(\mathbf{w}|\mathcal{D}) = \delta(\mathbf{w} - \hat{\mathbf{w}})$. In this case, the above integral reduces to the following plugin approximation:

$$p(\mathbf{y}|\mathbf{x}, \mathcal{D}) \approx \int p(\mathbf{y}|\mathbf{x}, \mathbf{w})\delta(\mathbf{w} - \hat{\mathbf{w}})d\mathbf{w} = p(\mathbf{y}|\mathbf{x}, \hat{\mathbf{w}}) \quad (10.109)$$

However, if we want to compute uncertainty in our predictions, we should use a non-degenerate posterior. It is common to use a Gaussian posterior, as we will see. But we still need to approximate the integral in Equation (10.108). We discuss some approaches to this below.

10.5.2.1 Monte Carlo approximation

The simplest approach is to use a **Monte Carlo approximation** to the integral. This means we draw S samples from the posterior, $\mathbf{w}_s \sim p(\mathbf{w}|\mathcal{D})$, and then compute

$$p(y = 1|\mathbf{x}, \mathcal{D}) \approx \frac{1}{S} \sum_{s=1}^S \sigma(\mathbf{w}_s^\top \mathbf{x}) \quad (10.110)$$

10.5.2.2 Probit approximation

Although the Monte Carlo approximation is simple, it can be slow, since we need to draw S samples *at test time* for each input $\mathbf{x}$. Fortunately, if $p(\mathbf{w}|\mathcal{D}) = \mathcal{N}(\mathbf{w}|\boldsymbol{\mu}, \boldsymbol{\Sigma})$, there is a simple yet accurate

deterministic approximation, first suggested in [SL90]. To explain this approximation, we follow the presentation of [Bis06, p219]. The key observation is that the sigmoid function $\sigma(a)$ is similar in shape to the Gaussian cdf (see Section 2.6.1) $\Phi(a)$. In particular we have $\sigma(a) \approx \Phi(\lambda a)$, where $\lambda^2 = \pi/8$ ensures the two functions have the same slope at the origin. This is useful since we can integrate a Gaussian cdf wrt a Gaussian pdf exactly:

$$\int \Phi(\lambda a) \mathcal{N}(a|m, v) da = \Phi\left(\frac{m}{(\lambda^{-2} + v)^{\frac{1}{2}}}\right) = \Phi\left(\frac{\lambda m}{(1 + \lambda^2 v)^{\frac{1}{2}}}\right) \approx \sigma(\kappa(v)m) \quad (10.111)$$

where we have defined

$$\kappa(v) \triangleq (1 + \pi v/8)^{-\frac{1}{2}} \quad (10.112)$$

Thus if we define $a = \mathbf{x}^\top \mathbf{w}$, we have

$$p(y = 1|\mathbf{x}, \mathcal{D}) \approx \sigma(\kappa(v)m) \quad (10.113)$$

$$m = \mathbb{E}[a] = \mathbf{x}^\top \boldsymbol{\mu} \quad (10.114)$$

$$v = \mathbb{V}[a] = \mathbb{V}[\mathbf{x}^\top \mathbf{w}] = \mathbf{x}^\top \boldsymbol{\Sigma} \mathbf{x} \quad (10.115)$$

where we used Equation (2.165) in the last line. Since Φ is the inverse of the probit function, we will call this the **probit approximation**.

Using Equation (10.113) results in predictions that are less extreme (in terms of their confidence) than the plug-in estimate. To see this, note that $0 < \kappa(v) < 1$ and hence $\kappa(v)m < m$, so $\sigma(\kappa(v)m)$ is closer to 0.5 than $\sigma(m)$ is. However, the decision boundary itself will not be affected. To see this, note that the decision boundary is the set of points $\mathbf{x}$ for which $p(y = 1|\mathbf{x}, \mathcal{D}) = 0.5$. This implies $\kappa(v)m = 0$, which implies $m = \bar{\mathbf{w}}^\top \mathbf{x} = 0$; but this is the same as the decision boundary from the plugin estimate. Thus “being Bayesian” doesn’t change the misclassification rate (in this case), but it does change the confidence estimates of the model, which can be important, as we illustrate in Section 10.5.1.

In the multiclass case we can use the **generalized probit approximation** [Gib97]:

$$p(y = c|\mathbf{x}, \mathcal{D}) \approx \frac{\exp(\kappa(v_c)m_c)}{\sum_{c'} \exp(\kappa(v_{c'})m_{c'})} \quad (10.116)$$

$$m_c = \bar{\mathbf{m}}_c^\top \mathbf{x} \quad (10.117)$$

$$v_c = \mathbf{x}^\top \mathbf{V}_{c,c} \mathbf{x} \quad (10.118)$$

where κ is defined in Equation (10.112). Unlike the binary case, taking into account posterior covariance gives different predictions than the plug-in approach (see Exercise 3.10.3 of [RW06]).

For further approximations of Gaussian integrals combined with sigmoid and softmax functions, see [Dau17].

10.6 Exercises

Exercise 10.1 [Gradient and Hessian of log-likelihood for multinomial logistic regression]

a. Let $\mu_{ik} = \text{softmax}(\boldsymbol{\eta}_i)_k$, where $\boldsymbol{\eta}_i = \mathbf{w}^T \mathbf{x}_i$. Show that the Jacobian of the softmax is

$$\frac{\partial \mu_{ik}}{\partial \eta_{ij}} = \mu_{ik}(\delta_{kj} - \mu_{ij}) \quad (10.119)$$

where $\delta_{kj} = I(k = j)$.

b. Hence show that the gradient of the NLL is given by

$$\nabla_{\mathbf{w}_c} \ell = \sum_i (y_{ic} - \mu_{ic}) \mathbf{x}_i \quad (10.120)$$

Hint: use the chain rule and the fact that $\sum_c y_{ic} = 1$.

c. Show that the block submatrix of the Hessian for classes c and c' is given by

$$\mathbf{H}_{c,c'} = - \sum_i \mu_{ic} (\delta_{c,c'} - \mu_{i,c'}) \mathbf{x}_i \mathbf{x}_i^T \quad (10.121)$$

Hence show that the Hessian of the NLL is positive definite.

Exercise 10.2 [Regularizing separate terms in 2d logistic regression †]

(Source: Jaakkola.)

a. Consider the data in Figure 10.15a, where we fit the model $p(y = 1|\mathbf{x}, \mathbf{w}) = \sigma(w_0 + w_1x_1 + w_2x_2)$. Suppose we fit the model by maximum likelihood, i.e., we minimize

$$J(\mathbf{w}) = -\ell(\mathbf{w}, \mathcal{D}_{\text{train}}) \quad (10.122)$$

where $\ell(\mathbf{w}, \mathcal{D}_{\text{train}})$ is the log likelihood on the training set. Sketch a possible decision boundary corresponding to $\hat{\mathbf{w}}$. (Copy the figure first (a rough sketch is enough), and then superimpose your answer on your copy, since you will need multiple versions of this figure). Is your answer (decision boundary) unique? How many classification errors does your method make on the training set?

b. Now suppose we regularize only the w_0 parameter, i.e., we minimize

$$J_0(\mathbf{w}) = -\ell(\mathbf{w}, \mathcal{D}_{\text{train}}) + \lambda w_0^2 \quad (10.123)$$

Suppose λ is a very large number, so we regularize w_0 all the way to 0, but all other parameters are unregularized. Sketch a possible decision boundary. How many classification errors does your method make on the training set? Hint: consider the behavior of simple linear regression, $w_0 + w_1x_1 + w_2x_2$ when $x_1 = x_2 = 0$.

c. Now suppose we heavily regularize only the w_1 parameter, i.e., we minimize

$$J_1(\mathbf{w}) = -\ell(\mathbf{w}, \mathcal{D}_{\text{train}}) + \lambda w_1^2 \quad (10.124)$$

Sketch a possible decision boundary. How many classification errors does your method make on the training set?

d. Now suppose we heavily regularize only the w_2 parameter. Sketch a possible decision boundary. How many classification errors does your method make on the training set?

Exercise 10.3 [Logistic regression vs LDA/QDA †]

(Source: Jaakkola.) Suppose we train the following binary classifiers via maximum likelihood.

- GaussI: A generative classifier, where the class-conditional densities are Gaussian, with both covariance matrices set to $\mathbf{I}$ (identity matrix), i.e., $p(\mathbf{x}|y = c) = \mathcal{N}(\mathbf{x}|\boldsymbol{\mu}_c, \mathbf{I})$. We assume $p(y)$ is uniform.
- GaussX: as for GaussI, but the covariance matrices are unconstrained, i.e., $p(\mathbf{x}|y = c) = \mathcal{N}(\mathbf{x}|\boldsymbol{\mu}_c, \boldsymbol{\Sigma}_c)$.
- LinLog: A logistic regression model with linear features.

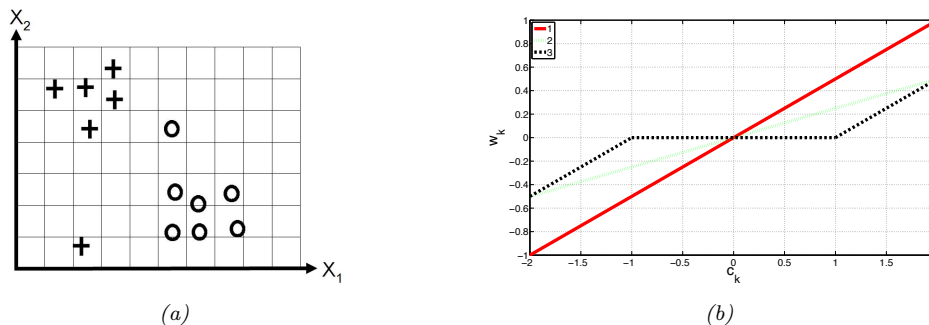


Figure 10.15: (a) Data for logistic regression question. (b) Plot of $\hat{w}_k$ vs amount of correlation c_k for three different estimators.

- d. QuadLog: A logistic regression model, using linear and quadratic features (i.e., polynomial basis function expansion of degree 2).

After training we compute the performance of each model M on the training set as follows:

$$L(M) = \frac{1}{n} \sum_{i=1}^n \log p(y_i | \mathbf{x}_i, \hat{\boldsymbol{\theta}}, M) \quad (10.125)$$

(Note that this is the *conditional* log-likelihood $p(y|\mathbf{x}, \hat{\boldsymbol{\theta}})$ and not the joint log-likelihood $p(y, \mathbf{x} | \hat{\boldsymbol{\theta}})$.) We now want to compare the performance of each model. We will write $L(M) \leq L(M')$ if model M *must* have lower (or equal) log likelihood (on the training set) than M' , for any training set (in other words, M is worse than M' , at least as far as training set logprob is concerned). For each of the following model pairs, state whether $L(M) \leq L(M')$, $L(M) \geq L(M')$, or whether no such statement can be made (i.e., M might sometimes be better than M' and sometimes worse); also, for each question, briefly (1-2 sentences) explain why.

- GaussI, LinLog.
- GaussX, QuadLog.
- LinLog, QuadLog.
- GaussI, QuadLog.
- Now suppose we measure performance in terms of the average misclassification rate on the training set:

$$R(M) = \frac{1}{n} \sum_{i=1}^n I(y_i \neq \hat{y}(\mathbf{x}_i)) \quad (10.126)$$

Is it true in general that $L(M) > L(M')$ implies that $R(M) < R(M')$? Explain why or why not.

